02 MAR 2026

AADK TASK 1

Design Your Own Data Model and Blueprint

MindMatrix

Concept Review & Notes

What is a class in Kotlin?

In Kotlin, a class is like a blueprint or template used to create objects. It defines the properties (data) and functions (behavior) that the objects created from it will have. It helps organize code in a structured way.

Difference between a class and an object

A class is just the design or blueprint. An object is the actual real-world instance created from that class.
For example, if Car is a class, then myCar is an object of that class.

What are primary and secondary constructors?

The primary constructor is defined in the class header and is used to initialize properties when the object is created.
A secondary constructor is defined inside the class body using the constructor keyword and can provide additional ways to create an object.

Purpose of init blocks and member functions

The init block runs immediately when an object is created. It is used to initialize or validate data.
Member functions define behaviors of the class. They allow objects to perform actions or calculate values.

Example of a simple data class

```
data class Student(
    val name: String,
    val age: Int,
    val grade: String
)
```

This data class automatically provides useful functions like toString(), copy(), and equals().

Design My Model

Chosen Domain: Recipe Manager App

I chose the Recipe Manager domain because recipes have clear properties like name, ingredients, cooking time, and favorite status. It is easy to relate to real life and helps practice working with lists and functions.

Class Design

```
class Recipe(
    val name: String,
    val ingredients: List<String>,
    val cookingTime: Int,
    val calories: Int,
    var isFavorite: Boolean = false
) {

    init {
        require(cookingTime > 0) { "Cooking time must be greater than 0"
}
        println("Recipe created: $name")
    }

    fun estimatedEffort(): String {
        return if (cookingTime < 20) "Easy"
        else if (cookingTime in 20..40) "Medium"
        else "Hard"
    }

    fun toggleFavorite() {
        isFavorite = !isFavorite
    }
}
```

Explanation of Design

Properties:

- name → Recipe name (String)
- ingredients → List of ingredients (List<String>)
- cookingTime → Time in minutes (Int)
- calories → Total calories (Int)
- isFavorite → Whether user marked it as favorite (Boolean)

Functions:

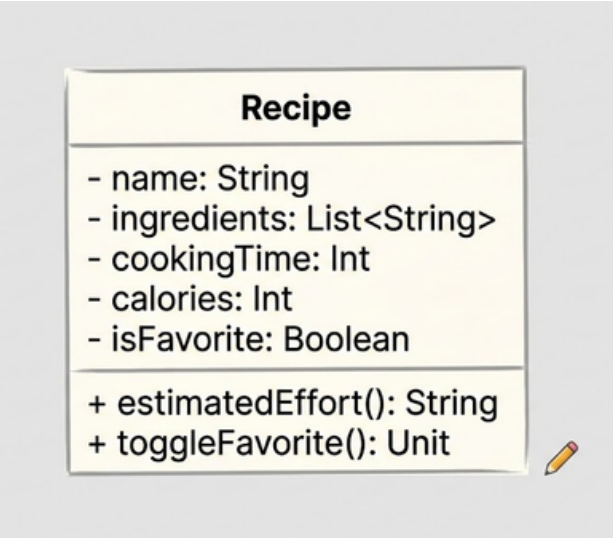
- estimatedEffort() → Returns difficulty level based on cooking time
- toggleFavorite() → Changes favorite status

Init block:

- Validates cooking time
- Prints recipe creation message

Diagram and Object Instances

UML-Style Diagram



This diagram shows the structure of the Recipe class.
The top section contains properties (data), and the bottom section contains methods (behavior).

Object Instances

Now I am creating two recipe objects using the Recipe class.

```
val recipe1 = Recipe(
    name = "Vegetable Pasta",
    ingredients = listOf("Pasta", "Tomato", "Capsicum", "Olive Oil"),
    cookingTime = 25,
    calories = 350,
    isFavorite = false
)

val recipe2 = Recipe(
    name = "Chocolate Mug Cake",
    ingredients = listOf("Flour", "Cocoa Powder", "Sugar", "Milk"),
    cookingTime = 10,
    calories = 450,
    isFavorite = true
)
```

Calling Functions on Objects

Here is a small Kotlin snippet showing how I can use the functions:

```
println(recipe1.estimatedEffort())   // Medium
recipe1.toggleFavorite()
println(recipe1.isFavorite)          // true
println(recipe2.estimatedEffort())   // Easy
recipe2.toggleFavorite()
println(recipe2.isFavorite)          // false
```

Explanation in simple words:

- estimatedEffort() checks cooking time and tells whether the recipe is Easy, Medium, or Hard.
- toggleFavorite() changes the favorite status from true to false or false to true.

This shows how objects created from the same class can store different data but still use the same functions.

Compose Integration Plan

In Jetpack Compose, I can display a list of Recipe objects using a LazyColumn. Each recipe can be shown inside a Card that displays the recipe name, cooking time, and estimated effort level.

I can add a favorite icon button inside each card. When the user clicks the button, it will call the toggleFavorite() function and update the UI using state management. If isFavorite is true, the icon can change color (for example, red heart), and if false, it can remain outlined.

This way, the UI will directly reflect the data from the Recipe class, and any changes made to the object will automatically update the screen.

Reflection

While designing the Recipe class, I understood how important it is to clearly define properties and functions based on real-world needs. It made me think about what information a recipe actually requires and what actions a user might perform, like marking it as favorite or checking difficulty level.
I also learned that adding validation in the init block helps maintain correct data and prevents errors. Designing the class first made everything more organized and structured.
Modeling data early helps in writing cleaner UI and logic code because the structure is already planned. When the data model is clear, it becomes easier to connect it with Compose UI components and manage state properly. It reduces confusion and makes the app more maintainable in the future.